Universität Konstanz

Introduction to Computing for the Social Sciences

Session 8 – Time Complexity

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning (8.01)

  K503, lecture 13:30-15:00, practice 15:15-16:45
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Time Complexity

Universität Konstanz

1. **Time complexity**
2. O-notation

It ran fine on the pilot

Universität Konstanz

MOTIVATING EXAMPLE

Your script processes the 1 000-row pilot in **four seconds**. The full dataset has 1 000 000 rows, a thousand times bigger. You start it on Friday expecting it on Saturday. It is still running three weeks later.

4 s
at n = 1 000

~1 hour
at n = 10^6 if the cost is linear

46 days
at n = 10^6 if it is quadratic

Nothing about the code changed. What property of it decides which of those two numbers you get?

Two students, the same answer, 20 000× apart

Universität Konstanz

MOTIVATING EXAMPLE

Both are asked to keep only the posts written by users in a list of 20 000. One writes a loop inside a loop. The other builds a lookup table first. **Both are correct and both hand in the same numbers.** On a million posts, one finishes in about **an hour and a half** and the other in **under a second**.

2×10^{10}
operations, nested loops

10^6
operations, with a hash set

20 000×
fewer, identical output

How do you *see* that difference by reading the code, before you run it?

Ten billion pages in a third of a second

Universität Konstanz

MOTIVATING EXAMPLE

A web search returns results from an index of well over **10^{10} pages** in about **0.3 seconds**. Looking at each page just once, even at a billion per second, would take **ten seconds**, and that is only to *look*, before any ranking.

10^{10}
pages indexed

0.3 s
to answer

~34
steps, if the cost is log n

No machine is fast enough to brute-force this. What class of procedure makes it possible, and what did it cost up front?

Recap: Selection sort

Procedure SELECTION-SORT(*A*, *n*)

Inputs:

A: an array

n: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For *i* = 1 to *n* - 1:

    smallest ← *i*

    For *j* = *i* + 1 to *n*:

        If *A*[*j*] < *A*[*smallest*]

            then *smallest* ← *j*

    swap *A*[*i*] with *A*[*smallest*]

Selection sort

- Efficiency of selection sort:
  - for a list of length n we have to search $n-1$ times for the minimum on a list of decreasing length
    - first search: $n-1$ elements
    - after first swap: $n-2$ elements
    - after second swap: $n-3$ elements, etc.
    - in total: $\sum_{i=1}^{n-1} i = \frac{(n-1)\,n}{2} \simeq \frac{n^2}{2}$
  - What does that mean?
    - sorting a list of **double** the size takes **four times** as long
    - sorting a list of **ten times** the size takes **100 times** as long

Complexity

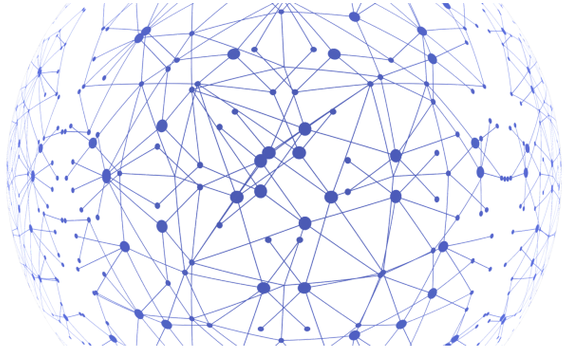
Universität Konstanz

What do we mean by complexity in Computer Science?

- **algorithmic complexity**
  - typically, refers to time complexity of a given algorithm
  - depends on a particular implementation
- **memory complexity**
  - required memory during calculation
  - mergesort: stack (due to recursion) and temporary second list
  - quicksort: optimal implementation only requires stack
- **problem complexity**
  - minimal effort needed to solve a problem
  - analyzed for typical cases
  - classification is independent of the specific algorithm – but depends on the algorithms known to solve a problem (more in Session 9)

A more general meaning of Complexity





- A **complicated** system has many pieces with specific functions and well-defined relationships. It has been carefully engineered or designed
- A **complex** system is composed of many particles that interact following some forces or dynamics. Its behavior follows from natural principles
- A program with lots of lines of code, functions, objects, etc. is *complicated*
- A problem with elements that depend on each other (e.g. a graph) can be *complex* – the time required to solve a problem depends on its complexity **and** our algorithms to solve it

Complexity

Universität Konstanz

- Why do we care about complexity?
  - problem complexity decides whether we can in principle solve a problem in a reasonable amount of time
  - algorithmic and memory complexity determine whether solutions are feasible to implement with the available memory of computers
    - example: run time estimates (algorithmic complexity)
      - home computer executes 10^8 operations per second
      - supercomputer executes 10^{12} operations per second

selection sort (n^2)

computer	thousand	million	billion
home	instant	2.8 hours	317 years
super	instant	1 second	1.6 weeks

mergesort ($n \log n$)

computer	thousand	million	billion
home	instant	0.2 sec	5 min
super	instant	instant	instant

Complexity

Universität Konstanz

- memory complexity and time complexity could be different
  - tradeoff between memory and time needs is often relevant in practice
- today we focus on *time complexity*
- time complexity varies in how it changes with the number of elements n
  - e.g., slowly rising, then sharp increase vs. sharp increase then slowly rising
  - makes certain algorithms more favorable for small n or for large n
- depending on data, complexity may vary; distinguish between
  - worst case complexity | average case complexity | best case complexity
- simplified analysis of an algorithm's efficiency
  - **approximate calculation of time complexity in terms of input size n**
  - machine independent, it is a property of the algorithm
  - time is measured in number of basic computer steps

Runtime Analysis

- The **runtime** sums up time units needed per instruction
  - **primitive operations**
    - assignments, comparisons, arithmetic operations, …
    - simplified measure as 1 time unit
  - **loops**
    - (number of iterations) × (time units per iteration)
  - **method calls**
    - time units for method
  - **recursive calls**
    - sum of all calls inside the recursion

Runtime Analysis Example: Selection Sort

Universität Konstanz

Algorithm in Python

```
1  def selectionsort(seq):
2      for i in range(len(seq) - 1):      # 1
3          k = i                          # 2
4          for j in range(i+1, len(seq)):# 3
5              if seq[j] < seq[k]:        # 4
6                  k = j                  # 5
7          seq[i], seq[k] = seq[k], seq[i]  # 6
8      return seq
```

1. range, get length & arithmetic operation = 3 time units
2. assignment = 1 time unit
3. range, arithmetic operation, get length = 3 time units
4. comparison = 1 time unit
5. assignment = 1 time unit
6. double assignment = 2 time units

Runtime Analysis Example: Selection Sort

Universität Konstanz

Algorithm in Python

```
1 def selectionsort(seq):
2     for i in range(len(seq) - 1):     # 1
3         k = i
4         for j in range(i+1, len(seq)):# 2
5             if seq[j] < seq[k]:
6                 k = j
7         seq[i], seq[k] = seq[k], seq[i]
8     return seq
```

1. loop with a maximum of $n - 1$ iterations
2. nested loop with a maximum of $n - 1$ iterations

Upper bound approximation

Runtime Analysis Example: Selection Sort

Universität Konstanz

Algorithm in Python

```
1 def selectionsort(seq):
2     for i in range(len(seq) - 1):      # 1
3         k = i                          # 2
4         for j in range(i+1, len(seq)):# 3
5             if seq[j] < seq[k]:        # 4
6                 k = j                  # 5
7         seq[i], seq[k] = seq[k], seq[i]  # 6
8     return seq
```

1. range, length & arithmetic = 3 time units
2. assignment = 1 time unit
3. range, arithmetic & length = 3 time units
4. comparison = 1 time unit
5. assignment = 1 time unit
6. double assignment = 2 time units

1. loop with a maximum of $n-1$ iterations
2. nested loop with a maximum of $n-1$ iterations

worst-case running time: $(n-1) \times \big(3+1+(n-1) \times (3+1+1)+2\big)$

Runtime Analysis Example: Selection Sort

1. range, get length & arithmetic operation = 3 time units
2. assignment = 1 time unit
3. range, arithmetic operation, get length = 3 time units
4. comparison = 1 time unit
5. assignment = 1 time unit
6. double assignment = 2 time units

1. loop with a maximum of $n-1$ iterations
2. nested loop with a maximum of $n-1$ iterations

$$(n-1) \times \big(3+1+(n-1) \times (3+1+1)+2\big) = 6n-6+5(n-1)^2 = 5n^2-4n-1$$

- the asymptotically dominant term is n^2
- constant terms and lower order terms asymptotically influence runtime only marginally: **O-Notation**

Average, best and worst running time

Universität Konstanz

- The **average** running time is the mean over all possible problem cases, assuming that they are all equally likely
- **Worst-case** running time: total maximum over any possible problem case
- **Best-case** running time: total minimum over any possible problem case

For sorting:

- Problem cases are possible initial orders of the list
- When selecting algorithms: we should have an idea of the distribution of problem cases to measure average running time
- We mostly care about the average but should check that the worst case is not too bad or too frequent

Example: Insertion sort

Procedure INSERTION-SORT(*A*, *n*)

Inputs:

A: an array; *n*: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For $i = 2$ to n:

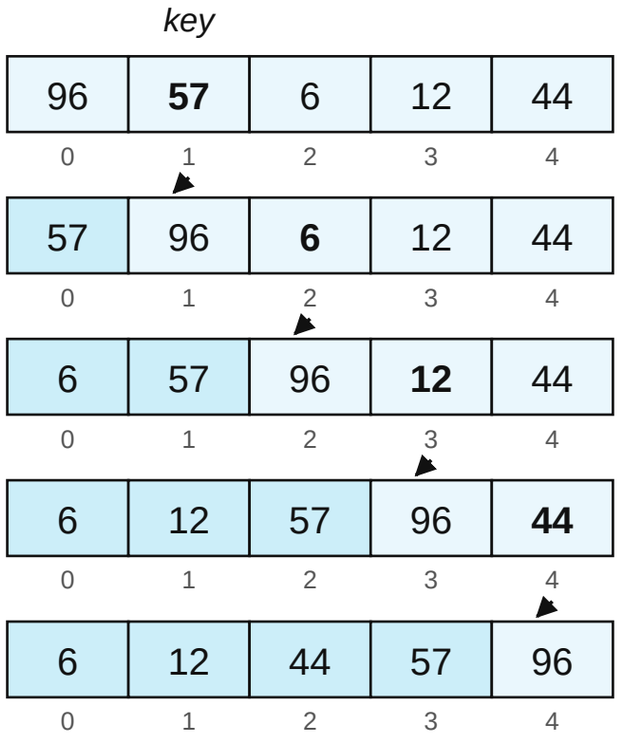
$key \leftarrow A[i]$; $j \leftarrow i - 1$

While $j > 0$ and $A[j] > key$:

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow key$



Rationale: incrementally sort by pushing value to the position where it belongs

Insertion sort: worst-case runtime analysis

Universität Konstanz

Procedure INSERTION-SORT(*A*, *n*)

Inputs:

A: an array; *n*: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For *i* = 2 to *n*:

key ← *A*[*i*]; *j* ← *i* - 1

While *j* > 0 and *A*[*j*] > *key*:

A[*j* + 1] ← *A*[*j*]

j ← *j* - 1

A[*j* + 1] ← *key*

Loop $n - 1$ times

Loop backwards potentially until $j = 0$: up to $n - 1$ iterations

worst-case running time: $(n-1) \times \left(2 + (n-1) \times (2+2) + 1\right)$ – grows with n^2

Insertion sort: best case

Procedure INSERTION-SORT(*A*, *n*)

Inputs:

A: an array; *n*: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For *i* = 2 to *n*:

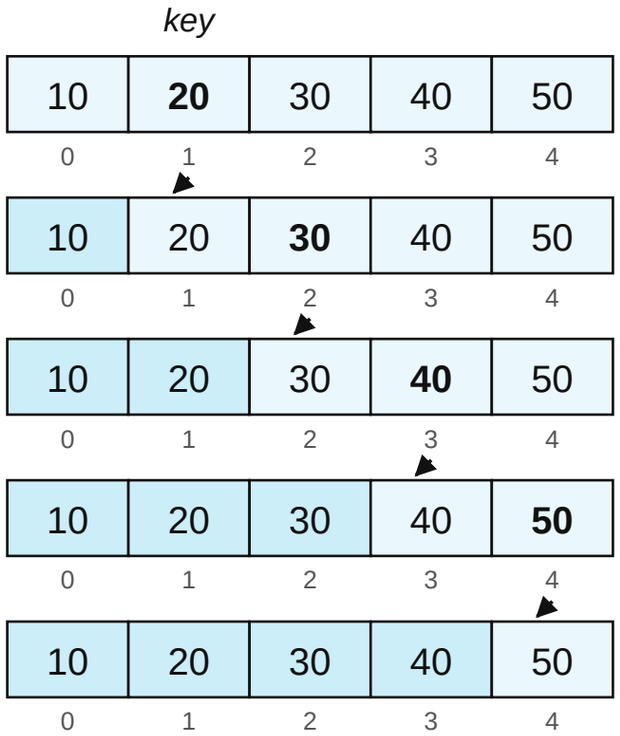
key ← *A*[*i*]; *j* ← *i* - 1

While *j* > 0 and *A*[*j*] > *key*:

A[*j* + 1] ← *A*[*j*]

j ← *j* - 1

A[*j* + 1] ← *key*



For an already sorted list: inner loop always stops at the first iteration

Insertion sort: best case

Procedure INSERTION-SORT(*A*, *n*)

Inputs:

A: an array; *n*: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For *i* = 2 to *n*:

key ← *A*[*i*]; *j* ← *i* - 1

While *j* > 0 and *A*[*j*] > *key*:

A[*j* + 1] ← *A*[*j*]

j ← *j* - 1

A[*j* + 1] ← *key*

- Average and worst-case: n^2
- Best case: n
  - Input is already sorted

Exercise: what is the best and worst-case runtime for quicksort?

Another example: Complexity of Mergesort

Procedure MERGE-SORT(A, p, r)

Inputs:

A: an array

p, r: the starting and ending index of the subarray

Result: $A[p..r]$ is sorted

If $p \geq r$

then return without doing anything

else $q \leftarrow \lfloor (p + r) / 2 \rfloor$

MERGE-SORT(A, p, q)

MERGE-SORT(A, $q + 1$, r)

MERGE(A, p, q, r)

- n is the length of A
- Time for best case: one operation
- Two recursive calls to arrays of length $n/2$
- Call to MERGE with an array of length n takes $c \cdot n$ steps, where c is some constant

$$T(n) = T(n/2) + T(n/2) + c \cdot n$$
$$= 2 \cdot T(n/2) + c \cdot n$$

Another example: Complexity of Mergesort

Universität Konstanz

Procedure MERGE-SORT(A, p, r)
Inputs:
A: an array
p, r: the starting and ending index of the subarray
Result: $A[p..r]$ is sorted
If $p \geq r$
then return without doing anything
else $q \leftarrow \lfloor (p + r) / 2 \rfloor$
MERGE-SORT(A, p, q)
MERGE-SORT(A, $q + 1$, r)
MERGE(A, p, q, r)

$$
\begin{aligned}
T(n) &= 2 \cdot T(n/2) + c \cdot n \\
&= 2 \cdot [2 \cdot T(n/4) + c \cdot n/2] + c \cdot n \\
&= 2^2 \cdot T(n/4) + c \cdot n + c \cdot n \\
&= 2^2 \cdot [2 \cdot T(n/8) + c \cdot n/4] + 2c \cdot n \\
&= 2^3 \cdot T(n/8) + 3c \cdot n \\
&\vdots \\
&= 2^k \cdot T(n/2^k) + k \cdot c \cdot n
\end{aligned}
$$

If k is enough for $n/2^k = 1$, then $k = \log_2 n$ and

$$T(n) = 2^{\log_2 n} + c \cdot n \log_2 n = n + c \cdot n \log_2 n$$

Another example: Complexity of BFS

Universität Konstanz

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

The size of a graph is measured by two quantities:

- Number of vertices $|V|$
- Number of edges $|E|$

Another example: Complexity of BFS

Universität Konstanz

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

The size of a graph is measured by two quantities:

- Number of vertices $|V|$
- Number of edges $|E|$
- We enqueue and dequeue each vertex exactly once

Another example: Complexity of BFS

Universität Konstanz

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

The size of a graph is measured by two quantities:

- Number of vertices $|V|$
- Number of edges $|E|$
- We enqueue and dequeue each vertex exactly once
- We check the adjacent nodes of each node. Each edge is checked exactly twice

Another example: Complexity of BFS

Universität Konstanz

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

The size of a graph is measured by two quantities:

- Number of vertices $|V|$
- Number of edges $|E|$
- We enqueue and dequeue each vertex exactly once
- We check the adjacent nodes of each node. Each edge is checked exactly twice

Running time of BFS grows with $|V| + 2|E|$

A mentor on algorithms: Manuel Blum



- Born in Venezuela, 1938
- CS professor at UC Berkeley and, since 2001, at Carnegie Mellon University
- Inventor of the k-minimum selection algorithm (and the calculation of its complexity)
- Turing Award for his contributions to the foundations of computational complexity theory and its application to cryptography and program checking
- Inventor of CAPTCHA as a Turing test

Known for being the mentor of very influential computer scientists: 3 Turing Awards, various Gödel and Knuth Prizes, 20 CS professors in top US universities

https://www.technologyreview.com/2023/10/24/1081478/manuel-blum-theoretical-computer-science-turing-award-academic-advisor/

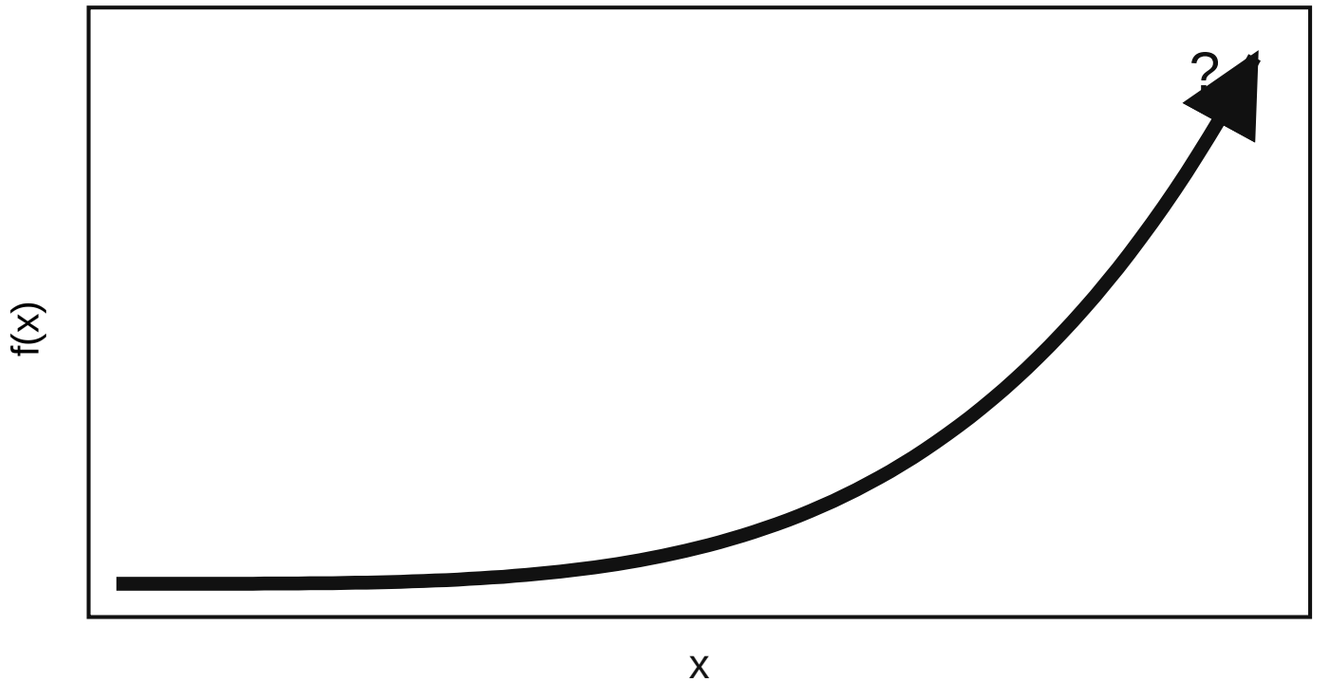
O-notation

1. Time complexity
2. **O-notation**

Asymptotic behavior of functions

The **asymptotic behavior** of a function $f(x)$ describes its value towards infinity

- values of $f(x)$ when $x \to \infty$ or $x \to -\infty$
- values of x where $f(x) \to \infty$ or $f(x) \to -\infty$



O-Notation

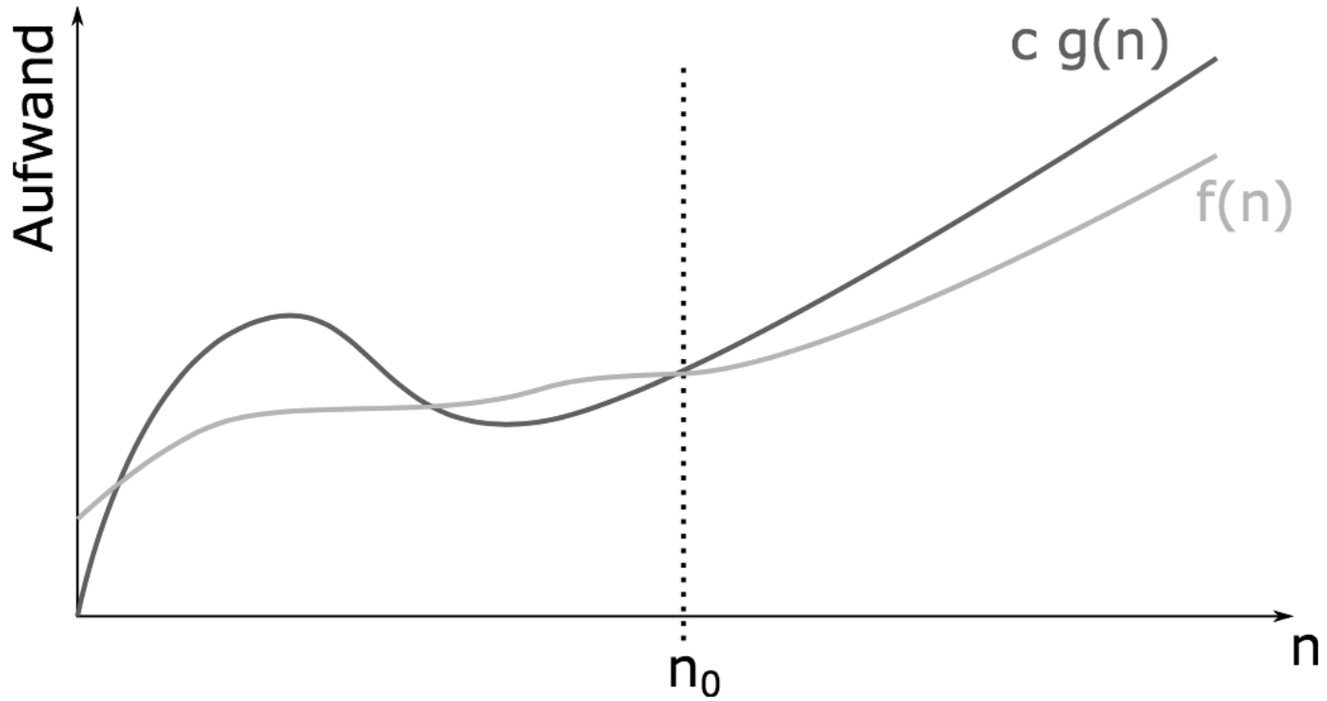
Universität Konstanz

Definition:

- O-Notation describes **classes of functions** with specific asymptotic properties
- also referred to as *Big O* notation (pronounced big-oh, Landau's symbol)
- values of a given function are after a given (large) value n_0
  - always larger or equal, or
  - always smaller or equal to a given function
- constants are not considered or expressed by an (arbitrary) chosen pre-factor
- separates into three different notations, where O is the most commonly used
  - O refers to the **upper bound**
  - Ω to the **lower bound**
  - Θ to the **tight bound**

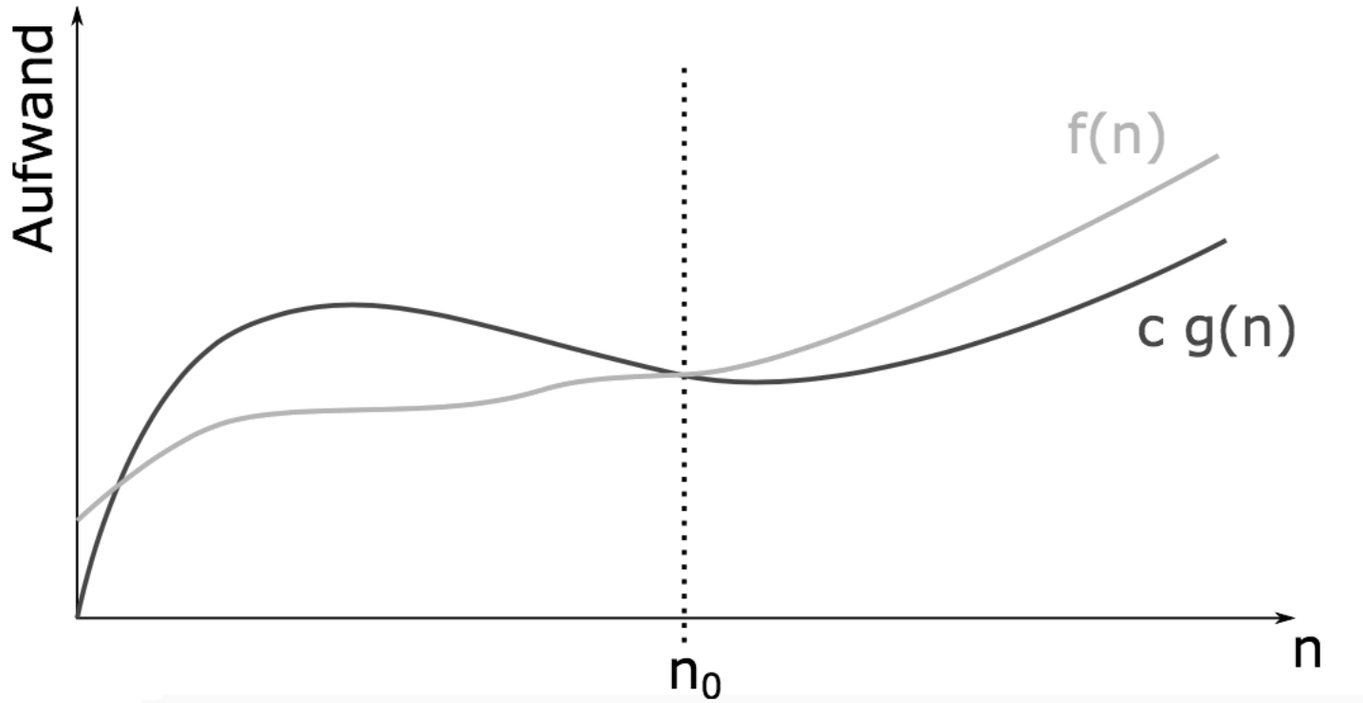
Upper Bound: O

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$$



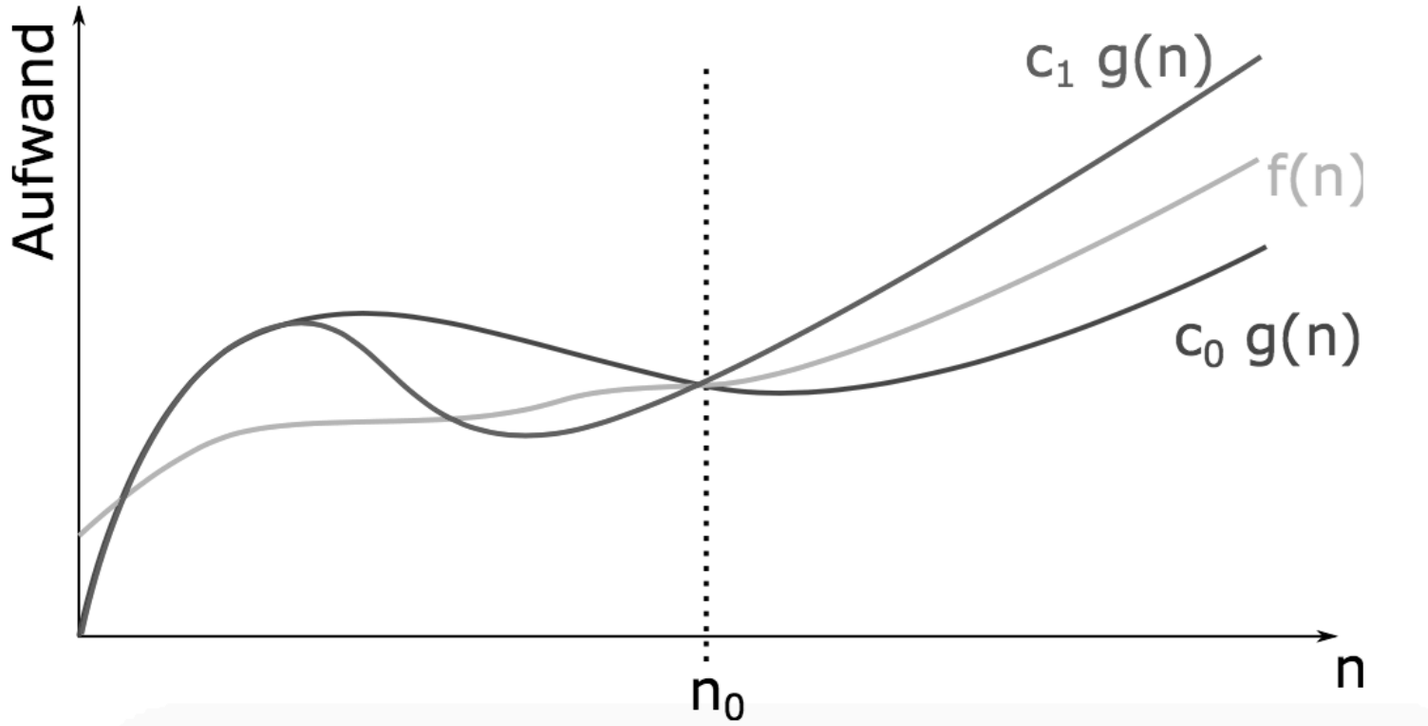
Lower Bound: Ω

$$\Omega(g(n)) = \{f(n) : \text{ there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$$



Exact Bound: Θ

$$\Theta(g(n)) = \{f(n) : \text{ there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .$$



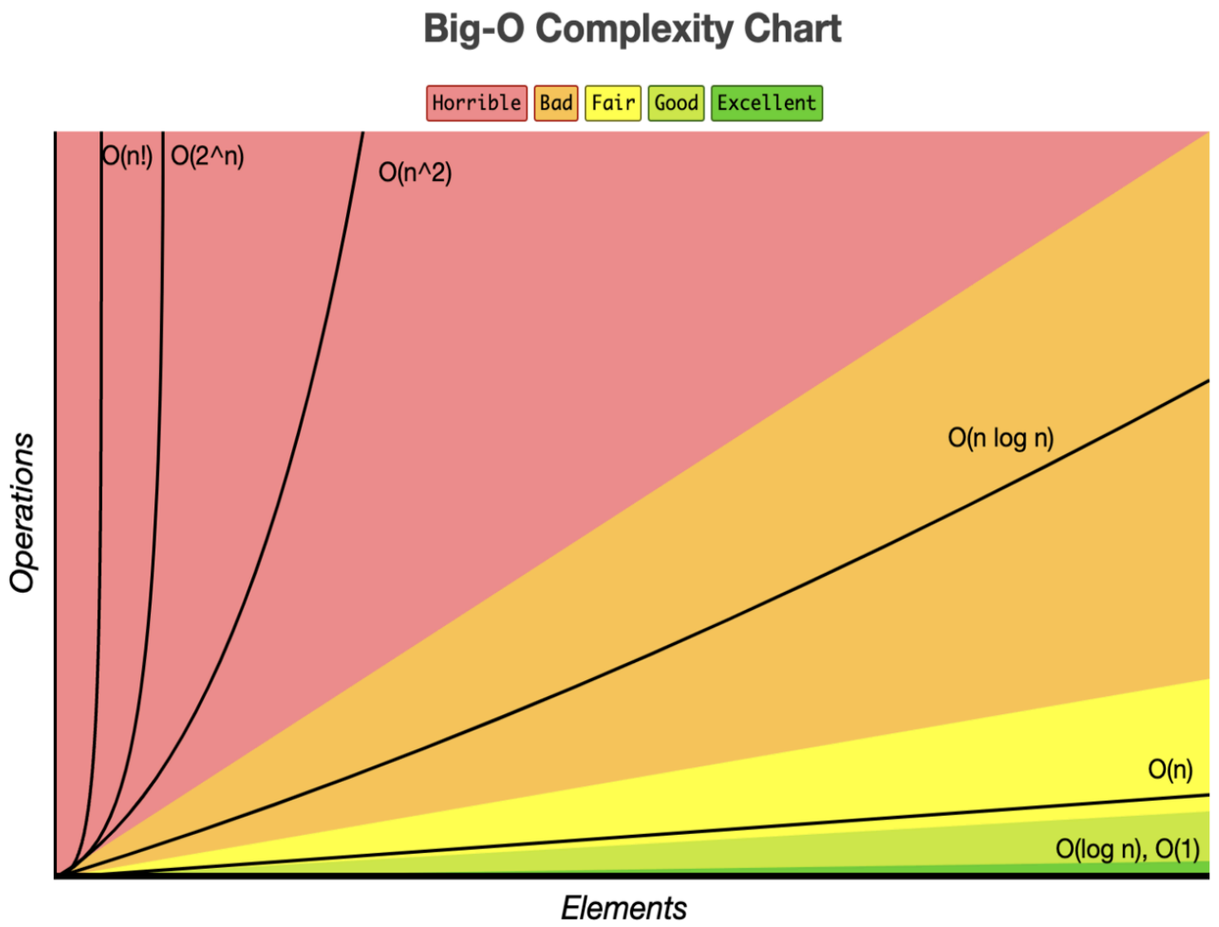
O-Notation

Use of the O-Notation for classification of functions:

- people often write $f = O(g)$ instead of $f \in O(g)$
  - this is a tolerated abuse of notation
- describes asymptotic upper, lower and exact bounds of a given function for large values
- interest is in **dominant terms** for large values only
  - more precise analyses are often very cumbersome and unnecessary
  - constant factors may often not be of interest
  - linear acceleration is typically easy to achieve (new hardware, …)
- goal: description of complexity through classes of functions
  - literally: “it can not get worse than f” if complexity is $O(f)$

O-Notation - General rules

- Ignore constants
- Certain terms “dominate” others
  - $O(1) \subset O(\log n) \subset O(n)$
  - $O(n) \subset O(n \log n) \subset O(n^2)$
  - $O(n^2) \subset O(2^n) \subset O(n!)$
  - i.e., ignore lower order terms



Growth of Functions

Universität Konstanz

n	1	10	100	1.000	10.000
$\log_{10} n$	0	1	2	3	4
$\log_2 n$	0	≈ 3	≈ 7	≈ 10	≈ 13
$\sqrt{n}$	1	≈ 3	10	≈ 32	100
n^2	1	100	10.000	1.000.000	100.000.000
n^3	1	1.000	1.000.000	1.000.000.000	1 Billion
2^n	2	1.024	$\approx 10^{30}$	$\approx 10^{301}$	$> 10^{3000}$
$1,1^n$	≈ 1	≈ 3	≈ 13.781	$\approx 2 \cdot 10^{41}$	$> 10^{414}$
$n!$	1	3.628.800	$\approx 9 \cdot 10^{157}$	...	...
n^n	1	10.000.000.000	10^{200}	10^{3000}	10^{40000}

O-Notation

- What does this imply?
  - remember our earlier examples:

selection sort $O(n^2)$

computer	thousand	million	billion
home	instant	2.8 hours	317 years
super	instant	1 second	1.6 weeks

mergesort $O(n \log n)$

computer	thousand	million	billion
home	instant	0.2 sec	5 min
super	instant	instant	instant

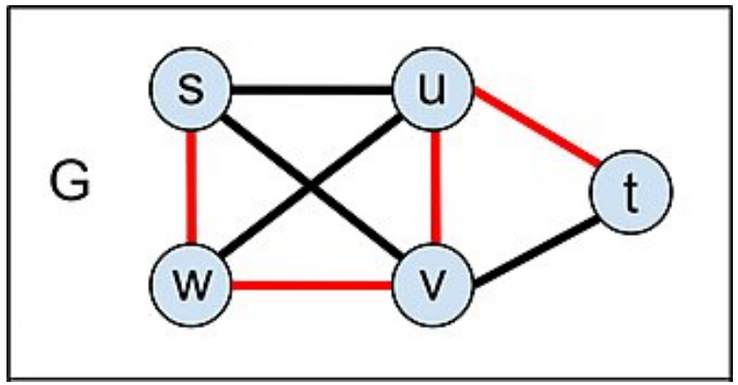
Using your laptop for an $O(n^2)$ algorithm will be unfeasible for n above a few million

Common use of O-Notation

Universität Konstanz

O-notation is a language to describe **upper bounds** on complexity

- The complexity of BFS is $O(|V| + |E|)$ because the runtime of BFS grows with $|V| + 2|E|$
- Mergesort is much better than selection sort because mergesort's complexity is $O(n \log n)$ and selection sort's complexity is $O(n^2)$
- Inverting matrices with the method we learn in high school is $O(n^3)$
- Testing if a number k is prime with a brute-force algorithm (dividing by all numbers up to $k - 1$) is $O(k)$
- Finding a Hamiltonian path in a graph with a brute-force algorithm (explore all paths) is $O(|V|!)$



Outtake – Social Science Applications

- Why do we care about complexity?
  - **Practical considerations**
    - algorithmic efficiency is often a precondition to doing any kind of data analysis in a reasonable time
    - understanding which implementation choices make an algorithm inefficient is thus basic tradecraft
    - there are often simple problems in day-to-day code that can lead to massive slow-downs
    - constants can matter, be cognizant of best case and average case (vs. worst case)
  - **“Hard” social science problems**
    - there are a number of rather common social science applications that have very high problem complexity
    - many of those examples are related to graphs/networks, e.g., the “traveling salesman” problem, graph isomorphisms

Summary

Universität Konstanz

- **Time complexity** counts basic steps as a function of the input size n; it is a property of the algorithm, not of the machine
- **Runtime analysis** adds up the cost of each instruction, multiplied by how often loops and recursive calls run
- Best, average and **worst case** can differ for the same algorithm (insertion sort: n against n^2)
- **O-notation** keeps only the dominant term: O is an upper bound, Ω a lower bound, Θ an exact bound
- Selection sort is $O(n^2)$, mergesort $O(n \log n)$, BFS $O(|V| + |E|)$: the growth rate decides what is feasible on large data

Next session: Theoretical Computer Science

ICSS (Theory and Practice) student feedback

Universität Konstanz

ICSS (Theory) student feedback

ICSS (Practice) student feedback

Appendix: About Practice Final Projects

Registering for exams

Universität Konstanz

- Exam registration period: 01.12.2025 - 15.01.2026 (deregistration possible until 31.01.2026)
- You need to register separately for ICSS (Theory) and ICSS (Practice)
- You also have to register for the Python Block Course to get the credits!
- Each year, you have one chance for ICSS (Practice)
  - The standard number of tries is two (thus two years)
- In ICSS (Theory) you will have to register for the resit if you need to take it
  - 2nd exam period: check university website
- Note for ICSS (Theory): if you pass the first try, you cannot go to the resit to improve your grade

Grading of ICSS (Practice)

Universität Konstanz

ICSS (Theory) and ICSS (Practice) are graded separately and it is possible to pass one and not the other.

You do not need to pass both parts the same year but we strongly recommend that you focus on passing both in the first semester.

Grading parts of ICSS (Practice):

- **Mandatory assignments:** To be allowed to submit a final project, you need to pass each of the four assignments
- **Final practical project.** The final grade of ICSS (Practice) is the grade of the final project

ICSS (Practice) final projects

Universität Konstanz

General aim of the final projects: individual students showcase their practical learning through a programming project of their own choice.

The **first aim** is to *build something* – the final product is the software, not the report. You will write papers in other courses.

The **second aim** is to *learn by doing*. You should go out of what is covered in the practical part and learn something new on your own. For example an API, a new library, another algorithm, a method…

The **third aim** is to develop the skill of *planning a project*. This is an iterative project, not a final exam. There will be plenty of chances to get feedback from lecturers and tutors, to discuss your ideas, feasibility, resources, etc.

Final project code

- Will be audited as part of the grading process
- Grading criteria
  - **Code readability:** variable names, structure, good programming practices…
  - **Code organization:** separation of project into files that structure the code and tasks
  - No redundancy, unnecessary functions or repeated code
  - Meaningful comments and justification/explanations
  - **Code execution**
    - Your code will be locally executed and inspected for bugs or unstable behavior
    - You have to include documentation on how to run it, dependencies, etc.
    - Target: a fellow student can install and run with 10 minutes of work

Final project report

Universität Konstanz

- Written report to complement code
- It should cover the following:
  - Brief motivation of the project: why should this be built?
  - Outline of design approach: structure, tasks, parts of the project
  - Detail on new algorithms, software, etc. Focus on what you learned and why it matters for the project
  - Demo, results, or outcomes of the project
  - Report on challenges, issues, further ideas
  - Not a research paper but it has to give context, motivation, and interpretation of the result/product of the project

Final project video

- Short video where you answer a few questions and show your code
- Designed not to invest a lot of time on production
- Think about it as an asynchronous interview
- Offers a channel for you to show insights that are not captured well in code, comments, or the report
- Instructions on what to show and questions will be released soon
- Questions are designed to show critical understanding of the project; if you abuse AI, it will be clear in the video
- Video has only pass/fail grade
  - Final grade is composed of code and report, weighting of each depends on project task and aims

Honor code and authorship statement

- A project has to be your own, nobody else should write a single line of code, report text, or content for the video
- You are not allowed to copy code from online or offline resources for the project, including AI coding assistants
- You will have to submit an authorship statement with your final project and abuse of AI coding assistants can lead to plagiarism investigations
- If you abuse AI or help from others, this will be visible in the quality of your code and of your video explanation

Final project submission

A final project submission is composed of:

1. project code
2. report
3. short video explanation
4. signed authorship statement

The deadline for the final project submission is **22.03, 23:59**

- Submission will be over github similarly as assignments
- If the video is too heavy as a file, it can be linked to the uni cloud

Timing: the last assignment deadline is 25.01. You have two months for exams and project work. Don’t underestimate the project, calculate a total of 90 hours of work on the project along the whole semester.

Random project ideas

Some (barely tested) examples of what you could do:

- A recommender website to choose where to live based on your interests and Facebook advertising data
  - Data collected with `pysocialwatcher`
  - Interactive website that collects user input and matches it to cities
- AWESOME-O: A program that generates movie arguments from a starting sentence and data from the Internet Movie DataBase
  - Training a simple language model with IMDB text
  - Program chatbot and experiment distinguishing real from fake
- Data collection and visualization of the Bluesky accounts of politicians of a country
  - Scraping a list of accounts from official sites
  - Using the Bluesky API
  - Network visualization and analysis

Random project ideas

Substantially extending an assignment:

- Training a model that identifies from a script whether the movie passes or not the Bechdel test
- Measuring polarization with Eurovision data and correlating that with interest rates of sovereign bonds
- Visualizing the correlation between voting results for US presidential candidates and Google trends across US states

Start thinking, talking, and asking questions soon! Do use the forum for random shower ideas, get our input soon and often!